

Day2 · 11차시

RAG 실전 — 인덱싱·검색·평가

데모는 10분, 프로덕션은 다른 문제.
10차시에서 붙인 RAG를, 오늘은 믿을 수 있게 만든다.

SECTION 00

오프닝 — 되는 데모 vs 믿을 수 있는 서비스

PDF 챗봇은 10분이면 된다. 문서가 100배가 되면 왜 무너지나?

“질문 3개 던져 그럴듯하면 OK? — 그건 신뢰성 보장이 아니다.”

— 10차시(왜 RAG) → 11차시(어떻게 잘 만드나) — 느낌 기반 테스트를 측정으로 바꾼다

지난 차시에서 이어서

▶ 긴 문서·많은 문서·어려운 질문에서도 무너지지 않게

10차시 — 왜 RAG

LLM은 기억 못 하고
내 데이터를 모른다
→ 검색으로 근거 공급

그때 만든 것

짧은 문장 5개
+ 코사인 검색
= ‘되는 데모’

오늘 — 어떻게 잘

수집·질의·스케일·평가
프로덕션 수준으로

“Day 2 문제” — 프로덕션 장벽

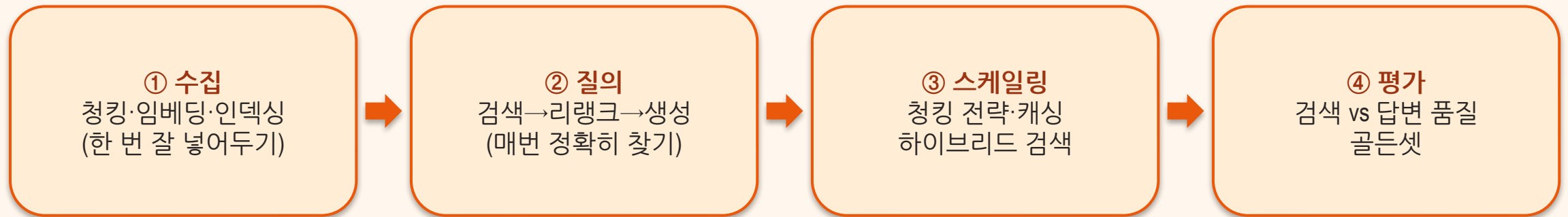
▶ ‘되는 데모’와 ‘믿을 수 있는 서비스’ 사이의 간극 — 오늘 이 벽을 다룬다

환각
엉뚱·없는 사실을
지어냄

정밀도↓
검색이 노이즈를
끌어옴

지연↑
규모에서 응답이
느려짐

이 차시 동안 얻는 것



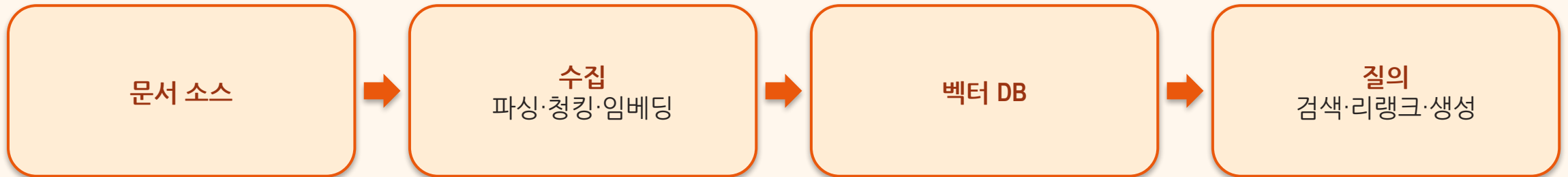
SECTION 01

수집(Ingestion) — 한 번 잘 넣어두기

수집은 데이터가 바뀔 때 한 번, 질의는 매 요청마다

두 개의 흐름으로 나뉜다

▶ 수집(한 번)과 질의(매번)는 실행 주기가 다르다 — 설계·최적화 포인트도 따로



수집 4단계 — 각각이 독립적 의사결정 지점

▶ 앞 단계가 망가지면 뒤가 전부 흔들린다 — 비정형 파일 파싱이 난이도의 핵심

1

파싱 (Parsing)

PDF·DOCX·HTML의 서식·잡정보를 걷어내고 본문만 — 스캔 문서는 OCR

2

청킹 (Chunking)

긴 세그먼트를 작은 의미 단위 조각(chunk)으로

3

임베딩 (Embedding)

각 청크를 의미를 담은 숫자 벡터로 변환

4

인덱싱 (Indexing)

임베딩+메타데이터를 벡터 DB에 — 빠른 검색용 자료구조

‘환불정책.pdf’ 한 장을 따라가 보자

▶ 같은 문서가 파싱→청킹→임베딩→인덱싱을 지나며 어떻게 변하는지

1

파싱 — 본문만 남긴다

머리글·쪽번호 제거 → “환불은 구매일로부터 14일 이내에만 가능하며... 단, 디지털 상품은 제외된다.”

2

청킹 — 의미 단위로 분할

청크1 “환불은 14일 이내, 영수증 지참” · 청크2 “디지털 상품·일부 할인 상품은 환불 제외”

3

임베딩 — 청크를 벡터로

청크1 → [0.021, -0.134, ..., 0.077] (1024차원 — 의미를 담은 숫자)

4

인덱싱 — 벡터 DB 레코드로

{id: “refund-01”, vector: [...], metadata: {source: “환불정책.pdf”, page: 3}}

왜 청킹이 필요한가

▶ ⚠ '긴 컨텍스트 = 무조건 좋다'는 착각

현실

대부분의 질문은 긴 문서의 **극히 일부**만 필요로 한다

비용·정확도

불필요하게 긴 컨텍스트는 비싸고, 오히려 정확도를 떨어뜨린다

청킹이란

세그먼트를 작은 **의미 단위** 조각으로 나누는 과정 (문자열 분할 **x**)

이유 3가지

컨텍스트 윈도우 한계 + 비용 + LLM 효과성

잘못된 청킹 = 정반대 답

▶ 모델보다 청킹·검색이 결과를 좌우한다

고정 글자 수로 무비판 분할

- 조항 A|B가 청크 경계로 쪼개짐
- 예외 조건이 다른 청크로 분리
- 검색이 본문만 끌어오고 예외 누락
- → 계약과 정반대 답

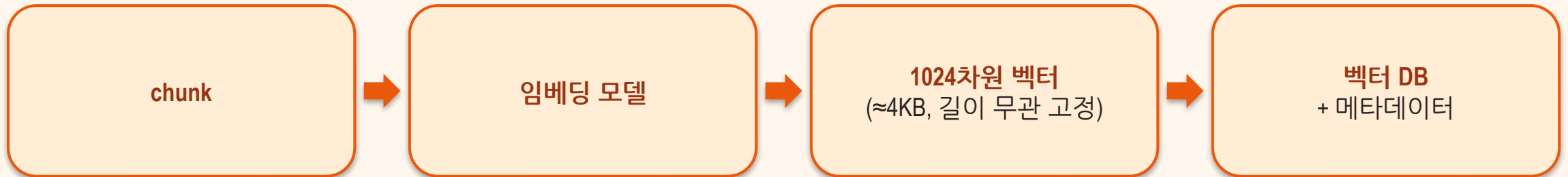
VS

의미 단위로 청킹

- 조항+예외를 한 청크에 유지
- 문장·섹션 경계 존중
- 검색이 완결된 의미를 끌어옴
- → 근거 있는 정확한 답

텍스트 → 의미 벡터 → 벡터 DB

▶ 'United States' ↔ 'USA'를 글자 비교로는 못 잡음 → 의미를 담은 숫자로



임베딩을 담는 그릇 — Vector DB 생태계

▶ 공통 개념은 같다: upsert(벡터+메타데이터) → top-k 질의 → 메타데이터 필터

Pinecone

완전 매니지드 SaaS
서버리스·자동 스케일
운영 부담 최소

Chroma

오픈소스·로컬 우선
pip 설치로 바로 사용
→ 오늘 실습에서 사용

Weaviate

오픈소스 + 클라우드
하이브리드 검색 내장

Qdrant

오픈소스 (Rust)
메타데이터 필터링 강력
고성능

pgvector

PostgreSQL 확장
이미 Postgres를 쓰면
추가 인프라 없이 도입

FAISS

Meta의 검색 라이브러리
DB가 아닌 인메모리 인덱스
연구·대규모 배치용

임베딩만으로 충분할까

▶ 둘을 합치는 게 하이브리드 검색(SECTION 03)의 근거

밀집 임베딩 (시맨틱)

- 의미를 벡터로 포착
- “환불” ↔ “며칠 안에” 매칭
- 유의어·바꿔쓴 표현에 강함

VS

키워드 색인 (보완)

- 역색인·**BM25**(Best Matching 25) — 글자 매칭
- BM25 = 단어 빈도 + 문서 길이 보정으로 점수화 — 키워드 랭킹의 표준 함수
- 처음 보는 고유명사·오타에 강함
- 임베딩의 보완 → **하이브리드**

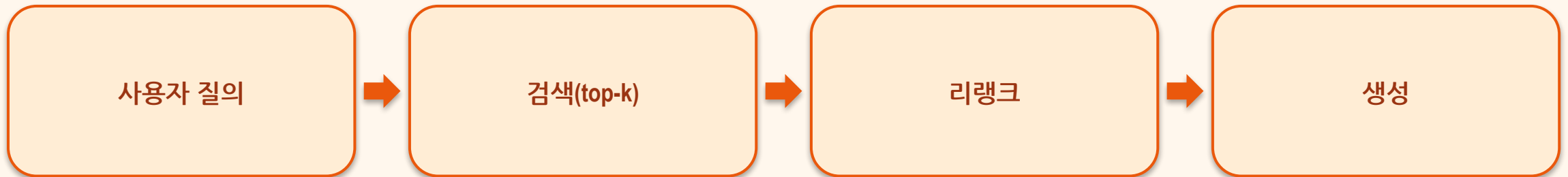
SECTION 02

질의(Query) — 매번 정확히 찾아 답하기

사용자 질의 → 검색 → 리랭크 → 생성 → 가드레일

질의 흐름 한눈에

▶ 질의 재작성으로 검색용 질의를 만들고 → 검색 → 리랭크 → 생성 → 가드레일로 검증



🔄 부족하면 재검색 — 답이 충분할 때까지

질의 하나를 따라가 보자 — “환불 규정으로 시 써줘”

▶ 각 단계에서 실제로 오가는 텍스트와 점수

1

질의 재작성

“환불 규정으로 시 써줘” → 명령(“시 써줘”) 제거 → 검색 질의 “환불 규정”

2

검색 top-3 (코사인 점수)

refund-01(0.62) · refund-02(0.58) · lunch-03(0.41 — 단어만 겹친 노이즈)

3

리랭크

cross-encoder가 질의+청크를 함께 읽고 lunch-03 탈락 → refund-01·02만 통과

4

생성

프롬프트: <문서>refund-01·02</문서> + 질문 → “환불은 14일 이내… (출처: 환불정책)”

검색엔 ‘맥락’만, ‘명령’은 빼라

▶ 사용자 질의 = 명령 + 검색용 맥락 — 검색엔 맥락만 넣는다

그대로 검색 ⚠

- “2025년 여행으로 시 써줘” 통째로 검색
- 명령(“시 써줘”)이 검색 벡터를 오염
- 엉뚱한 시·무관 문서를 끌어옴

VS

재작성 후 검색

- 맥락만 추출: “2025년 여행”
- = 검색 질의(retrieval query)
- 관련 문서를 정확히 매칭

시맨틱 검색 vs 키워드 검색

▶ 기본 스택과 실습은 시맨틱 — 규모에서 합치는 하이브리드는 SECTION 03

시맨틱(밀집) 검색

- 질의를 벡터로 → 벡터 DB 유사도 탐색
- 의미 매칭 (USA ↔ United States)
- 유사도 = 점곱(dot product)

VS

키워드(희소) 검색

- **TF-IDF**(단어 빈도 × 역문서 빈도)·**BM25**(Best Matching 25)
— 글자 매칭
- 어휘(lexicon) 공간에서 동작
- 오타·희귀 고유명사·정확 매칭에 강함

단위 벡터끼리는 점곱 = 코사인 유사도

▶ numpy 행렬 곱 한 번이면 전체 청크와의 유사도가 다 나온다

dot

두 벡터의 내적
= 방향 정렬 정도

$\|v\| = 1$

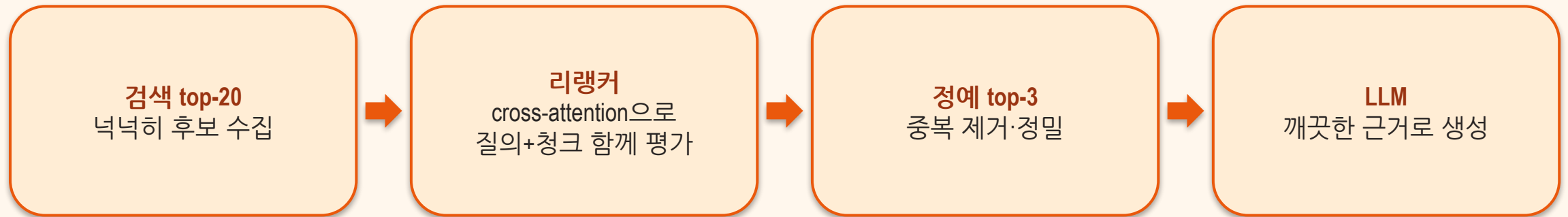
임베딩은 정규화된
단위 벡터

= cos

점곱이 곧
코사인 유사도

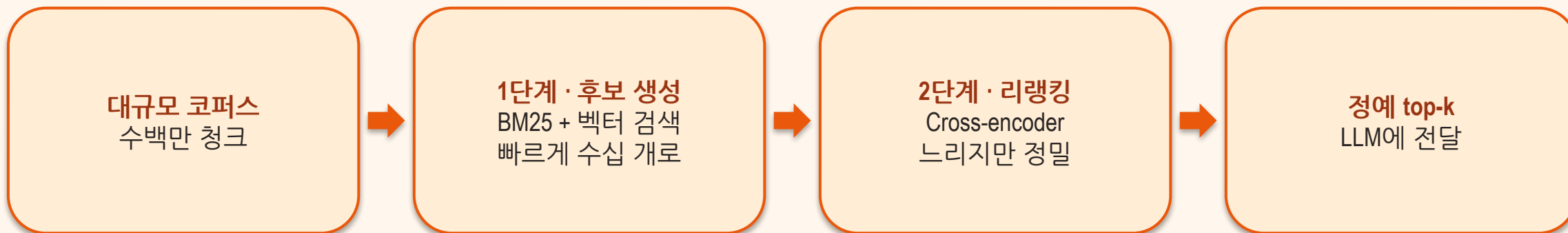
top-k가 곧 최적은 아니다

▶ 유사도 상위만 뽑으면 ① 중복 ② 어텐션 미활용 — 리랭커는 질의+청크를 함께 평가



2단계 검색 파이프라인 — 속도와 정확도의 역할 분담

▶ 1단계는 빠르고 넓게(후보), 2단계는 느리고 정밀하게(선별)



검색된 청크 '만' 근거로 답하기

▶ 생성: 검색 청크를 컨텍스트로만 / 가드레일: 청크에 충실한지(환각) 검증

```
시스템 = """아래 <문서>의 내용만 근거로 답하라.  
문서에 없으면 '모른다'고 답하고, 답에 출처를 함께 표기하라."""
```

```
user = f"<문서>\n{검색된_청크}\n</문서>\n\n질문: {query}"
```

- 출처/인용을 함께 내도록 지시 → 검증 가능
- 가드레일: 답이 청크에 충실한지, 편향·유해성은 없는지

SECTION 03

스케일링 — 규모에서 무너지는 곳

문서 1만 개에서 멀쩡하던 게 100만 개에서 깨진다

무엇이 동시에 커지나

▶ 청크가 많을수록 top-k에 정답을 넣기 어렵고 노이즈가 신호를 압도한다

문서 양
수십만~수백만
→ 인덱싱·저지연 유지 난항

문서 복잡도
수천 페이지
→ 파싱·청크 폭증

쿼리 부하
QPS↑
→ 수평확장·캐싱·rate limit

청크는 작게? 크게?

▶ 정답은 데이터마다 다르다 — 고정 크기·문장 단위·시맨틱 청킹을 측정하며 고른다

작은 청크

- 검색이 정밀(핵심만)
- 맥락이 부족할 수 있음
- 조각 수↑ → 인덱스·연산↑

VS

큰 청크

- 맥락이 풍부
- 노이즈·토큰 낭비↑
- 정답이 잡정보에 묻힐 위험

캐싱 & 동적 모델 라우팅

▶ 진짜 비용 = LLM 입력 토큰(쿼리+컨텍스트 2K~4K) — 캐싱을 총총이 + 요청별 라우팅

1

응답 캐싱

같은 질문 → LLM 호출 없이 즉시 응답

2

체크 캐싱

검색 결과 재사용 — 벡터 DB 조회 절감

3

임베딩 캐싱

같은 텍스트를 다시 임베딩하지 않음

4

동적 모델 라우팅

단순 요청은 저가 모델 · 복잡한 작업만 프리미엄 추론 모델

같은 질문이 다시 오면 무슨 일이 일어나나

▶ 캐시는 위층부터 검사 — 위에서 맞으면 아래 연산을 전부 생략

1

응답 캐시 HIT

“환불은 며칠 이내?”가 또 옴 → 정규화된 질의가 캐시 키 → 검색·LLM 0회, 저장된 답 즉시 반환

2

체크 캐시 HIT

“환불 기간은?”(표현만 다른 질의) → 검색 결과(refund-01...)는 재사용, 생성만 새로

3

임베딩 캐시 HIT

문서 100개 중 1개만 수정 → 나머지 99개 체크는 해시 동일 → 임베딩 API 호출 생략

4

전부 MISS → 라우팅

“영수증 없이 환불?”(단순) → 저가 모델 / “정책 A·B 충돌 분석”(복잡) → 프리미엄 모델

시맨틱 + 키워드 — 점수를 융합한다

▶ 규모에서 단순 벡터 검색만으론 정확도가 무너진다 — 두 검색을 병렬로 돌려 합친다



동적 데이터는 전체 재인덱싱 대신 증분 업데이트로 인덱스를 유지.

질의 “B73973 주문 환불” — 융합이 순위를 바꾼다

▶ 고유명사(주문번호)는 임베딩이 처음 보는 토큰 — 키워드 검색이 살린다

Dense만 (벡터 검색)

- “B73973”은 학습에 없던 토큰 → 의미 벡터가 못 잡음
- 1위: 환불정책 일반 조항 (0.55)
- 정답 “주문 B73973 처리 이력” 청크는 **8위** → top-3 밖
- → 일반론만 답하거나 환각

VS

Dense + Sparse 융합 (RRF)

- BM25가 “B73973”을 **정확 매칭** → sparse 1위
- RRF 점수 = $1/(60+\text{dense순위}) + 1/(60+\text{sparse순위})$
- 정답 청크: $1/(60+8) + 1/(60+1)$ → **융합 1위**
- → 해당 주문의 처리 이력으로 정확히 답변

SECTION 04

RAG 평가 — “느낌 기반 테스트”를 버려라

평가 없이는 개선도 없다. 무엇을, 어떻게 측정하나

검색 품질 vs 답변 품질

- ▶ ★ 검색기를 LLM보다 먼저 최적화 — 재현율 40%면 프롬프트를 아무리 다듬어도 실패

검색 품질 (retrieval)

- 올바른 청크를 찾았나
- 비유: 좋은 사서
- 지표: $\text{recall@k}(\text{재현율}) \cdot \text{precision@k}(\text{정밀도}) \cdot \text{nDCG}$

VS

답변 품질 (generation)

- 찾은 걸로 정확히 답했나
- 비유: 신뢰할 만한 저자
- 둘 다 + 협업까지 평가

재현율 vs 정밀도

▶ 검색이 무너지면 전체가 무너진다 — 가장 치명적인 건 낮은 재현율

낮은 재현율 (놓침)

- 정답 청크를 못 끌어옴(완전/부분)
- → “모른다” 하거나 **환각**
- 가장 치명적

VS

낮은 정밀도 (노이즈)

- 노이즈 청크를 끌어옴
- 단어만 겹치는 무관한 청크
- 필요한 사실이 없음

검색이 완벽해도 생성에서 실패한다

▶ 진단 핵심 — Faithful한데 틀림 = 검색기 문제 / Faithful하지 않음 = LLM(생성) 문제

Faithfulness 실패

= 환각. 답이 청크(문서)에 근거하는가가 **Faithfulness** — 청크와 모순·근거 없는 답이 가장 파괴적

컨텍스트 활용 실패

준 청크의 일부만 반영(순서 편향) → 불완전한 답

Relevance 실패

사실은 맞지만 질문 의도를 빗나감 — “안전한가요?”에 기능만 나열

어디서 무너졌나 — 3계층 진단

▶ 증상을 보고 어느 계층부터 고칠지 정한다 — 위에서 아래로 점검

수집 실패

OCR 오류·보일러플레이트 혼입·인코딩 깨짐 → 쓰레기가 들어가면 쓰레기가 나온다

검색 실패

낮은 재현율(놓침)·낮은 정밀도(노이즈) → 하이브리드·리랭킹으로 완화

생성 실패

환각·활용 실패·관련성 실패 → 프롬프트·가드레일·모델 교체로 완화

검색 지표의 사다리 — 기초에서 순위 인식으로

▶ 이름은 어려워도 질문은 하나 — ‘정답 청크가 몇 등으로 나왔나’

기초

recall@k(재현율 — top-k가 정답을 놓치지 않았나) · **precision@k**(정밀도 — 가져온 k개가 얼마나 맞나)

MRR

Mean Reciprocal Rank(평균 역순위) — 첫 정답 순위의 역수(1등=1, 3등=1/3)를 질문 전체로 평균

nDCG

normalized Discounted Cumulative Gain(정규화 할인 누적 이득) — 관련도 높은 청크가 위일수록 점수↑, 순위가 내려갈수록 할인(discount)

UMBRELA

UMbreला is the **B**ing **RE**levance **A**ssessor(재귀 약어) — 정답 라벨 없이 LLM이 관련도 0~3 자동 채점, Bing 방식의 오픈소스 재현 — 골든셋 부담↓

골든셋으로 반복 측정하기

▶ 오프라인(배포 전) 평가가 오늘의 초점 — 운영 중엔 가벼운 온라인 평가를 얻는다

1

골든셋 작성

질문 + 기대 근거/정답 모음 — 반복 가능한 자동 채점의 기준

2

파이프라인 실행

각 질문을 RAG에 넣어 검색 청크 + 생성 답 수집

3

채점

키워드 포함 / ROUGE-L·BERTScore / LLM-as-a-Judge + Rubric — 다음 장에서 자세히

4

비교·개선

청킹·검색 바뀌가며 점수 변화 측정 → 개선 반복

③채점 자세히 — 표면 유사도 → 의미 유사도 → 판단

▶ 아래로 갈수록 정교하고 비싸다 — 실무는 셋을 섞어 쓴다 (싼 지표로 넓게, Judge로 깊게)

키워드 포함

정답 키워드("14일")가 답에 있는지 — 가장 단순·공짜. 오늘 실습의 기본 채점

ROUGE-L

정답↔생성 답의 **최장 공통 부분수열(LCS)** 비율 — 글자 겹침만 봐서 "14일 이내"↔"2주 안"을 다른 답으로 판정

BERTScore

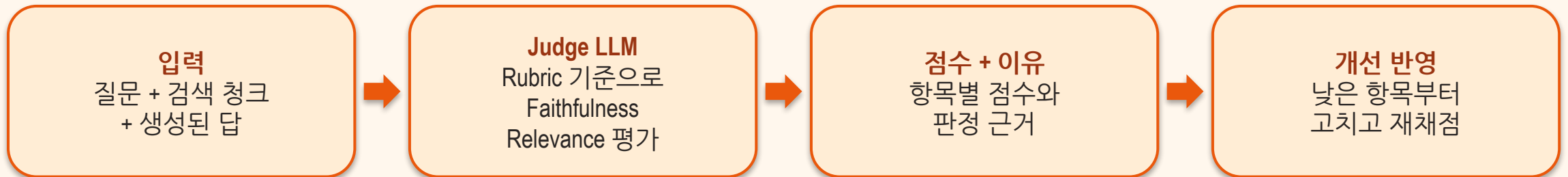
두 답의 토큰을 **임베딩으로 매칭**한 의미 유사도 — 바꿔 쓴 표현은 잡지만, 문서에 근거했는지(Faithfulness)는 못 봄

LLM-as-a-Judge

채점 기준표(Rubric)를 Judge LLM에 주고 **Faithfulness·Relevance**를 점수+이유로 — 유사도가 못 보는 것을 본다

LLM-as-a-Judge — Judge LLM에 Rubric을 쥐여준다

▶ Rubric = 채점 기준표. ROUGE·BERTScore가 못 잡는 Faithfulness·Relevance를 채점한다



㉵ ⚠ Self-preference bias — 같은 모델 계열로 채점하면 후해진다 → Judge는 다른 모델로

Judge LLM에 실제로 주는 프롬프트

▶ 실습 STEP 15에서 이 프롬프트를 그대로 실행한다

```
RUBRIC = """<문서>와 <답변>을 보고 두 항목을 1~5로 채점하라.  
- Faithfulness: 답변이 <문서> 내용에만 근거하는가 (지어낸 것 없음 = 5)  
- Relevance: 답변이 질문 의도에 맞는가 (의도를 정확히 충족 = 5)  
형식: Faithfulness: N / Relevance: N / 이유: 한 줄"""
```

- Judge 호출은 **temperature=0** — 채점이 흔들리지 않게
- Judge는 다른 모델 계열로 — self-preference bias 회피
- 점수만이 아니라 **이유**를 받아야 어디를 고칠지 보인다

SECTION 05

정리

오늘 배운 것을 세 줄로

오늘 3줄

- RAG = 수집(한 번) + 질의(매번) — 청킹은 문자열 분할이 아니라 의미 단위 분할
- 품질은 모델보다 청킹·검색이 좌우 — 검색기를 LLM보다 먼저 최적화
- 평가 없이는 개선 없다 — 골든셋으로 반복 측정, Judge는 다른 모델로